

Easy Track – Basic Implementation

- Implement **Linear Search**
- Implement **Binary Search** (array must be sorted)
- Focus on correct logic and working output (Pseudocode already discussed in class)

Medium Track – Modular Programming

- Implement **Linear Search** using a **function**
 - Input: array, key
 - Output: index / -1
- Implement **Binary Search** using a **function**
 - Same structure: parameters + return value

★ Focus on **functions**, clean design, and **reusability**

Bonus Challenge – Counterfeit Coin Problem

You have 8 identical-looking coins.

One coin is **counterfeit** and has a **different weight** (heavier OR lighter – you don't know which).

- Left side heavier
- Right side heavier
- Both sides equal

🎯 Your Task

- 1 Identify which coin is counterfeit
- 2 Use the **minimum** number of **weighings**
- 3 **Bonus:** Also determine if it is heavier or lighter

Write your weighing strategy step by step – logic matters more than code here!

Goal of the lab: Clear thinking + clean implementation 🚀

• EASY

1. Find a Peak Element

Write a function to find any peak element in an array. where an element is a peak if it is not smaller than its neighbors.

2. Binary Search on Descending Array:

Modify binary search to find the index of a *key* in an array sorted in decreasing order.

• MEDIUM

1. Count Occurrences in Sorted Array:

Using binary search, write a function to count how many times a number *key* appears in a sorted array.

Think about the worst case and the best case analysis

• DIFFICULT

1. Integer Square Root (Binary Search on Numbers)

Given an integer n , find the floor of the square root of n $\sqrt{n}$ using binary search.

2. Find a Peak Element

An element in an array is a peak if it is not smaller than its neighbors. Implement a function to find and return any peak element index in $O(\log n)$

● Easy Track (Attempt ANY TWO)

- Implement Bubble Sort.
- Implement *Selection Sort*.
- Implement *Insertion Sort*.

● Medium Track (Attempt BOTH)

- Improve Bubble Sort to stop early if the array is already sorted.
- Improve Selection Sort to avoid unnecessary swaps.

● Difficult Track (Attempt BOTH)

1. **Modify Bubble Sort** to track the last swap position and *shrink the range for the next pass*.
2. You are given n distinct numbers. Finding the largest and second largest separately may take nearly $2n$ comparisons.

How can you find both using *fewer comparisons*?
Give the **maximum number of comparisons required by your method** (exact value in terms of n).

Hint: Think of the numbers as players in a knockout tournament.